



Tecnicatura Universitaria
en Programación

PROGRAMACIÓN IV

BACK END

Anexo N°2:

Diferenciación entre Reverse Proxy,
BFF y API Gateway

Material de Estudio
2° Año – 4° Cuatrimestre



Índice

Introducción	2
Reverse Proxy	2
BFF (Backend for Frontend)	2
API Gateway	3
¿Por qué se confunden?	3
¿Pueden convivir estos tres patrones?	4

Introducción

Cuando hablamos de microservicios, siempre aparece la misma pregunta: **¿Cómo hacemos para que el cliente** (ya sea una app web, un celular o un sistema externo) **pueda comunicarse con un conjunto de servicios que viven detrás de la arquitectura?**

Si exponemos directamente cada microservicio, caemos en problemas: **demasiados puntos de acceso, contratos diferentes, manejo disperso de seguridad y dificultad para evolucionar**. Para resolver esto, históricamente aparecieron distintas soluciones, que hoy a veces se confunden entre sí: **reverse proxy, BFF y API gateway**.

La confusión nace porque **todos se ubican “delante” de los microservicios y median entre el cliente y el backend**. Pero sus propósitos y alcances son muy diferentes.

Reverse Proxy

El primer escenario es el más sencillo: **tenemos muchos servicios y queremos un único punto de entrada que se encargue de redirigir cada petición al lugar correcto**. Aquí aparece el reverse proxy.

Un **reverse proxy** como **Nginx** no sabe nada del negocio. **Su trabajo es infraestructura pura**: balancear carga, ocultar direcciones internas, manejar certificados SSL, comprimir respuestas y, en algunos casos, cachear contenido estático.

Podemos pensar en él como el portero de un edificio: recibe a todas las visitas en la puerta, verifica a qué departamento van y las redirige al ascensor correspondiente. Pero nunca participa de la reunión dentro del departamento: solo facilita la llegada.

BFF (Backend for Frontend)

Ahora imaginemos otra situación: una aplicación móvil necesita mostrar el “perfil del usuario”. Para eso hay que consultar tres microservicios distintos: usuarios, pedidos y notificaciones. Si la app tuviera que llamar a los tres, la complejidad se dispararía: más latencia, más contratos que mantener, más código en el cliente.

Aquí entra el **BFF** (Backend for Frontend). Su misión es **adaptar la arquitectura a las necesidades específicas de un canal**. El BFF se ubica entre el frontend y los microservicios, **actuando como traductor: recibe una petición simple y devuelve un resultado ya consolidado, listo para mostrarse en pantalla**.

En la práctica, un **BFF** suele implementarse en **Node.js** porque permite manejar de forma ágil muchas peticiones concurrentes y transformar datos rápidamente. Lo clave es entender que el **BFF** sí puede tener lógica de negocio ligera, pero siempre orientada a mejorar la experiencia del frontend, no a centralizar reglas del dominio.

API Gateway

El tercer escenario va más allá: necesitamos un punto único no solo para enrutar, sino también para gobernar. El **API Gateway es ese componente que combina enrutamiento con seguridad, monitoreo y resiliencia**.

A diferencia del reverse proxy, que solo se ocupa de la infraestructura básica, el **API Gateway decide si una petición pasa o no**: valida tokens, aplica límites de tráfico, transforma requests, abre o corta circuitos frente a fallos, y deja trazas para el monitoreo.

Podemos imaginarlo como la recepción principal de una empresa: no solo indica hacia dónde ir, sino que controla quién entra, en qué condiciones y qué permisos tiene. En este sentido, **frameworks** como **Spring Cloud Gateway** o **Kong** se utilizan para implementar estas funcionalidades críticas en arquitecturas de microservicios.

¿Por qué se confunden?

Porque a primera vista, los tres hacen lo mismo: están delante de los microservicios y median entre cliente y backend.

La diferencia está en el propósito:

- El **reverse proxy** es puramente técnico, no sabe de negocio.
- El **BFF** es un traductor hecho a medida del frontend.
- El **API Gateway** es el punto central de control y seguridad de toda la arquitectura.

Diferencias clave

Característica	Reverse Proxy	BFF	API Gateway
Nivel	Infraestructura	Lógica de adaptación	Infraestructura + gobernanza
Rol principal	Enrutamiento y optimización	Adaptar datos al frontend	Punto central de control y seguridad
Lógica de negocio	No	Solo la necesaria para UI	No (aunque gestiona reglas transversales)
Orientación	Genérico	Por canal (web, mobile, IoT)	Global (todo el sistema)
Ejemplo de tecnologías	Nginx, HAProxy	Node.js, Express, NestJS	Spring Cloud Gateway, Kong, Zuul

¿Pueden convivir estos tres patrones?

A primera vista podría parecer que debemos elegir entre un **reverse proxy**, un **BFF** o un **API Gateway**. Sin embargo, en la práctica, **los tres cumplen roles tan distintos** que no solo pueden convivir, sino que **suelen complementarse** de manera natural en arquitecturas maduras.

Imaginemos una ciudad:

- El **reverse proxy** es como la autopista que conecta las entradas principales, regulando el tráfico y evitando embotellamientos.
- El **API Gateway** sería la aduana central, donde se revisa la identidad de quien entra y se aplican reglas generales de seguridad.
- El **BFF** es el guía turístico que traduce y adapta la experiencia según el tipo de visitante: uno necesita un recorrido rápido, otro quiere detalles históricos, y otro busca solo lo esencial.

De esta forma, cada uno tiene un rol definido:

- **Reverse proxy** en la frontera, manejando balanceo, cacheo, enrutamiento y SSL.
- **API Gateway** como punto central de control, aplicando reglas de seguridad, monitoreo y resiliencia.
- **BFFs específicos** para cada canal (web, mobile, IoT), simplificando y adaptando la información que necesitan.

La clave está en no intentar que uno haga el trabajo de otro. Juntos, forman una cadena de responsabilidad clara y eficaz.

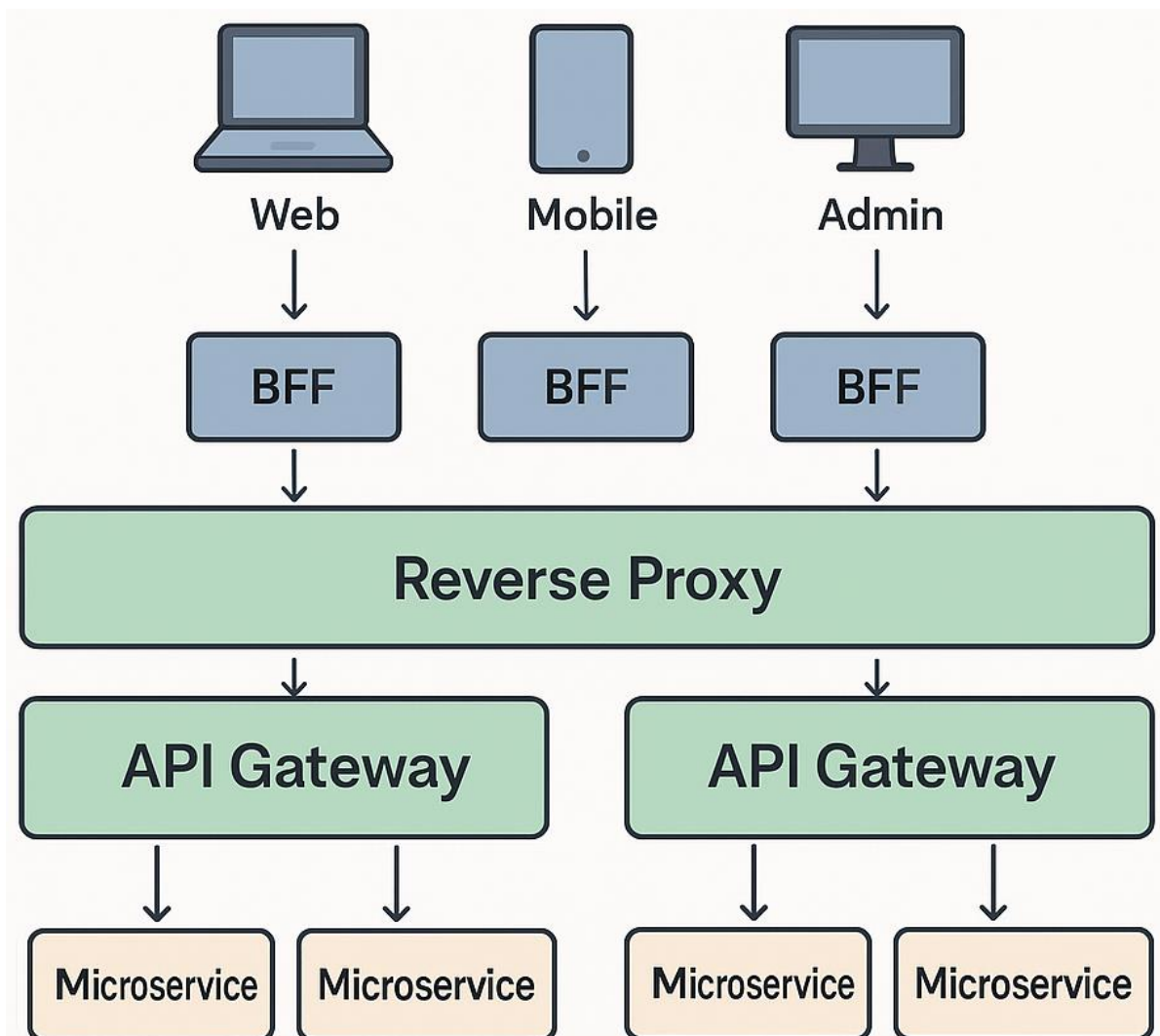


Imagen 1: Elaboración Propia.



Atribución-No Comercial-Sin Derivadas

Se permite descargar esta obra y compartirla, siempre y cuando no sea modificado y/o alterado su contenido, ni se comercialice. Universidad Tecnológica Nacional Facultad Regional Córdoba (S/D). Material para la Tecnicatura Universitaria en Programación, modalidad virtual, Córdoba, Argentina.